Reproducing FinMem on a Leakage-Free Window

Implementation Development Report

MBA Data-Science Seminar — reproduction & critical assessment of
Yu et al., *FinMem: A Performance-Enhanced LLM Trading Agent with Layered Memory and Character Design* (arXiv:2311.13743)

Dan Shoshan & Nimrod Sagi

Code: github.com/Doubledanx2/seminarion_DS_Finmem_implementation

1. Executive summary

We reproduced the FinMem LLM trading agent on the authors' own codebase and re-ran the entire experiment on a **leakage-free 2026 test window** (2026-01-02 to 2026-06-01) that postdates every model's knowledge cutoff — the fix for the paper's central flaw, where its 2022–23 test window sat inside the backbone LLM's training data. We built the full data pipeline (Alpaca news, SEC filings, Gemini summarisation, FinBERT sentiment), assembled a corrected and extended configuration we call **FinMem-Ours**, and benchmarked it against three baselines under one audited accounting convention.

Headline result. On out-of-sample data the layered-memory + persona apparatus did not add value — it was beaten by simpler baselines:

Strategy	Mean cum. return (0 bps)	Mean Sharpe
No-memory ablation	+1.7%	+0.38
LC-Trader (plain long-context)	–2.9%	+0.16
Buy & Hold	–4.1%	+0.11
FinMem-Ours (full apparatus)	–5.3%	–0.15

Both stripped-down baselines — removing memory (the ablation) and a plain long-context model fed the same news (LC-Trader) — outperformed the complete system, which even trailed passive Buy & Hold. This is the core of our critical assessment.

2. Objective & experimental design

The paper's reported out-performance may reflect the backbone LLM *remembering* 2022–23 prices rather than predicting them. Our design therefore forces every generative component to have a knowledge cutoff before the data window: decision model **gpt-4.1-mini** (cutoff Jun-2024) and summariser **Gemini 3.1 Flash-Lite** (Jan-2025); train 2025-07-01 to 2025-12-31, test 2026-01-02 to 2026-06-01.

Tickers: TSLA, NFLX, AMZN, MSFT, COIN (the paper's five). Hyper-parameters were **frozen at the paper's published values before any test run** (the git commit hash is recorded) so that nothing was tuned on the test data, and the comparisons were declared in advance: FinMem-Ours vs Buy&Hold, vs the no-memory ablation, vs LC-Trader, and vs the original as-shipped artefact.

3. What we built — the data pipeline

• **News:** downloaded 18,311 articles across the 5 tickers from the Alpaca historical news API (full monthly coverage; single-source Benzinga feed, disclosed as a limitation).
• **Filings:** pulled the 10-K/10-Q MD&A sections from SEC EDGAR / sec-api.io, dated by publication date.
• **Summarisation:** condensed all news and filings with Gemini 3.1 Flash-Lite, each item summarised independently from its own text only.
• **Sentiment:** scored every news item with FinBERT, locally on an RTX 3090.
• **Prices:** yfinance adjusted close. The pipeline outputs one model-input file per ticker, which we validated by stepping it through the paper's market-environment day by day.

4. What we built — the FinMem-Ours system

FinMem-Ours is the paper's architecture made to actually match the paper's description, frozen at a single git commit. The concrete things we implemented or corrected:

• **Self-adaptive persona** — the agent switches between a risk-seeking and a risk-averse stance based on its recent 3-day return (the paper describes this; the shipped code only ever used the static risk-seeking line).
• **Deep-memory retention** — we changed the memory mechanics so the deep layer actually keeps information over time (as shipped it discarded everything within a few days and never retained a single SEC

filing).
• **Extended reflection** — a daily self-review of the last week's decisions, written into long-term memory (described in the paper, absent from the code).
• **Filing seeding** — the latest annual and quarterly report are loaded into memory on day one, with their true filing dates.
• **Long-only positions** — we hold at most one share and never short (the shipped portfolio allowed costless short-selling); local ada-002 embeddings; a 7-day return signal. Smaller corrections to the research code (sentiment labels, crashes on empty news days, a Python-3.12 port) are catalogued in the repository log.

5. Baselines we ran

Every strategy is scored the same way (next section). We compared FinMem-Ours against:

• **Buy & Hold** — stay fully invested.
• **No-memory ablation** — the identical model and prompt, but memory retrieval returns nothing, so the agent decides from the current day's information alone.
• **LC-Trader** — a plain long-context model fed the same news stream each day (no persona, no memory, no retrieval, no sentiment) that simply decides buy / sell / hold. This isolates the question: in a modern long-context world, does FinMem's machinery beat just giving a plain model the same news? We used prompt-caching to keep its cost low ($2.47 total).

6. How we measured performance

All strategies use one accounting convention: **long-only unit positions, simple daily returns compounded over the full price series including the final test day**, reported with and without 10 bps transaction costs. Mid-project an independent recomputation disagreed with our first numbers; we traced it to two mistakes in our own reporting code — a dropped final trading day and an accidental short position whenever the agent said 'sell' — fixed both, and added an automatic check that the Buy & Hold return equals the raw end-to-end price change (the run fails otherwise). Every number in this report uses the corrected convention; significance is assessed with the Wilcoxon signed-rank test and a bootstrap CI.

7. Results (test window 2026 H1)

Ticker	Strategy	CR 0bps	CR 10bps	Sharpe	Sortino	MaxDD	Turn	%Long
TSLA	FinMem-Ours	–23.1%	–25.5%	–1.92	–2.57	–30%	31	54%
TSLA	No-memory	–5.5%	–8.0%	–0.37	–0.47	–26%	27	43%
TSLA	LC-Trader	–7.9%	–8.1%	–0.33	–0.61	–24%	3	98%
TSLA	BuyHold	–5.1%	–5.2%	–0.14	–0.25	–24%	1	100%
TSLA	As-shipped	–9.2%	–10.2%	–0.50	–0.82	–27%	11	80%
NFLX	FinMem-Ours	+3.8%	+1.9%	+0.44	+0.48	–15%	18	55%
NFLX	No-memory	+19.8%	+17.5%	+1.56	+1.36	–13%	20	39%
NFLX	LC-Trader	+2.1%	+1.2%	+0.32	+0.49	–21%	9	96%
NFLX	BuyHold	–5.6%	–5.7%	–0.19	–0.30	–21%	1	100%
AMZN	FinMem-Ours	+15.5%	+13.3%	+1.67	+2.56	–13%	19	67%
AMZN	No-memory	+24.5%	+22.1%	+2.74	+4.15	–7%	19	54%
AMZN	LC-Trader	+15.3%	+15.2%	+1.31	+2.15	–20%	1	100%
AMZN	BuyHold	+15.3%	+15.2%	+1.31	+2.15	–20%	1	100%
MSFT	FinMem-Ours	–3.9%	–5.5%	–0.20	–0.20	–21%	17	72%
MSFT	No-memory	–6.0%	–8.2%	–0.56	–0.37	–16%	23	42%
MSFT	LC-Trader	–3.6%	–4.1%	–0.12	–0.15	–27%	5	98%
MSFT	BuyHold	–2.2%	–2.3%	–0.00	–0.00	–26%	1	100%
COIN	FinMem-Ours	–18.5%	–20.5%	–0.70	–1.02	–28%	24	57%
COIN	No-memory	–24.2%	–25.9%	–1.49	–1.62	–25%	23	37%
COIN	LC-Trader	–20.7%	–20.9%	–0.35	–0.66	–45%	3	99%
COIN	BuyHold	–22.8%	–22.9%	–0.44	–0.82	–45%	1	100%

Pooled Wilcoxon (daily): FinMem-Ours vs Buy&Hold p=0.69 (not significant); FinMem-Ours vs No-memory p=0.075 (memory hurt).
CR = cumulative return; MaxDD = maximum drawdown; Turn = number of position changes; %Long = share of days invested.

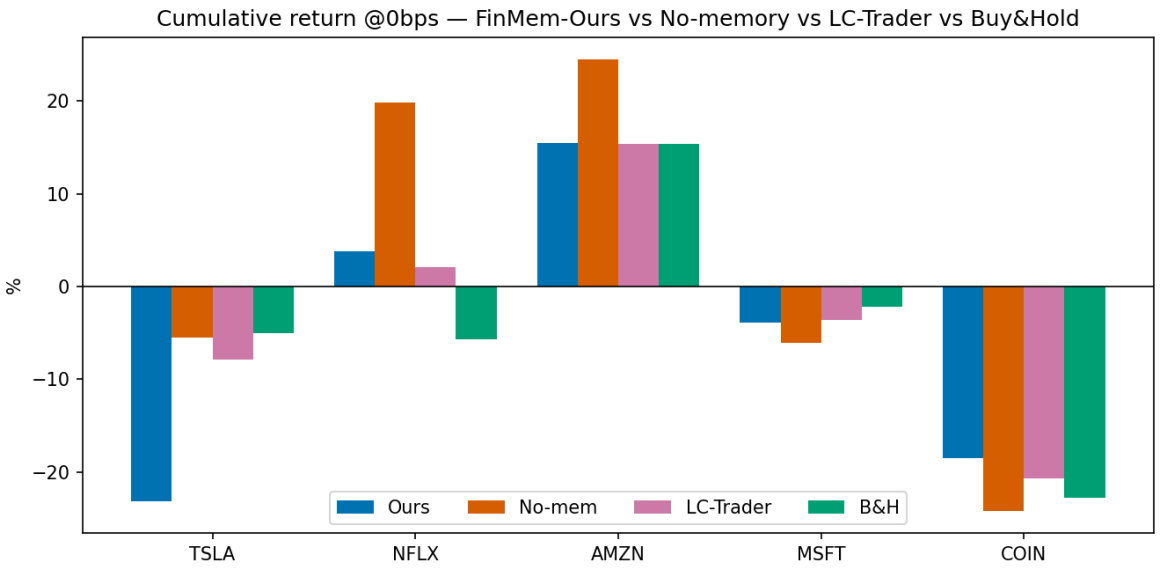

Fig 1. Cumulative return by ticker, all four strategies (0 bps).

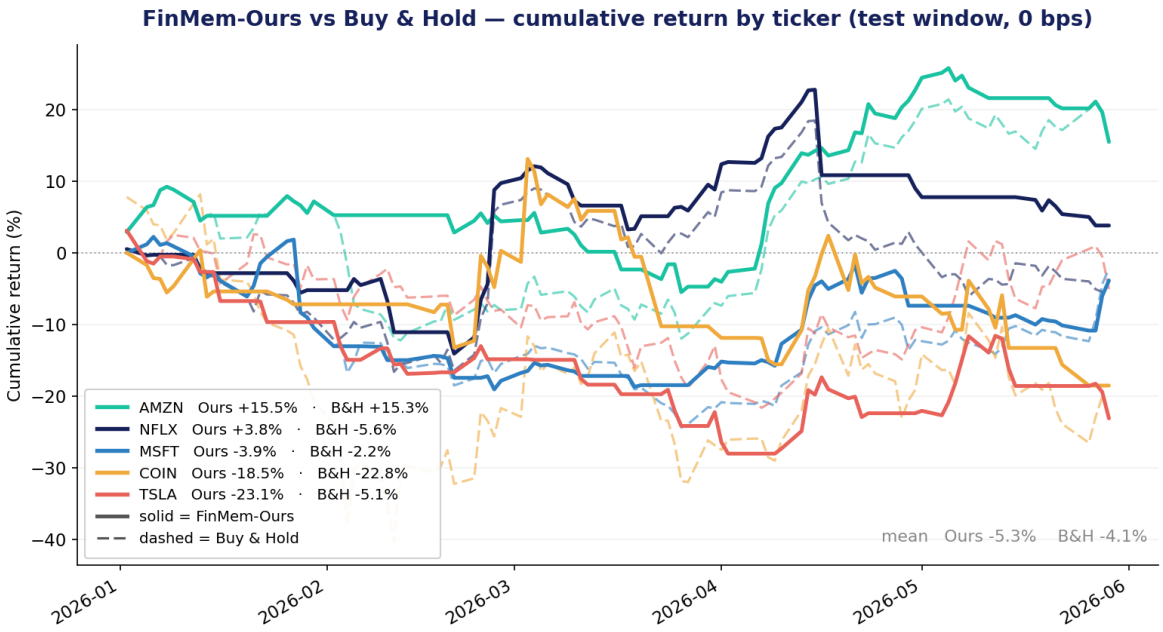

Fig 2. FinMem-Ours (solid) vs Buy & Hold (dashed) per ticker.

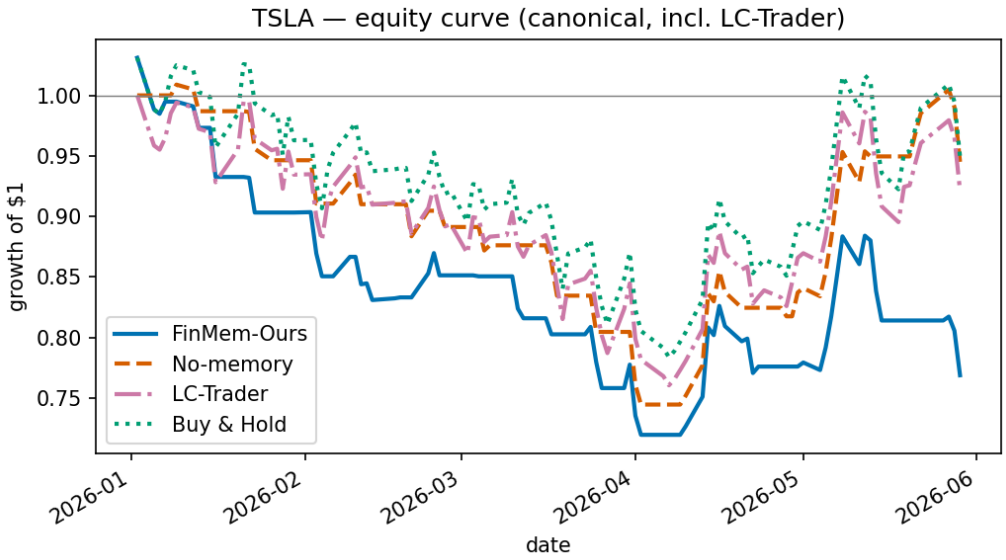

Fig 3. TSLA equity curves — FinMem-Ours is the lowest line.

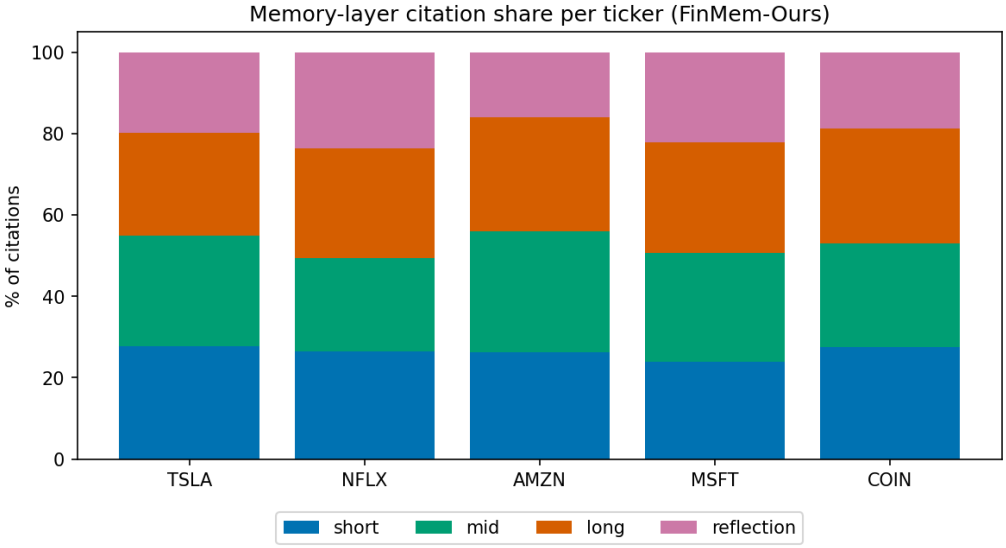

Fig 4. Memory-layer citation share — the agent does draw on its deep / reflection memory.

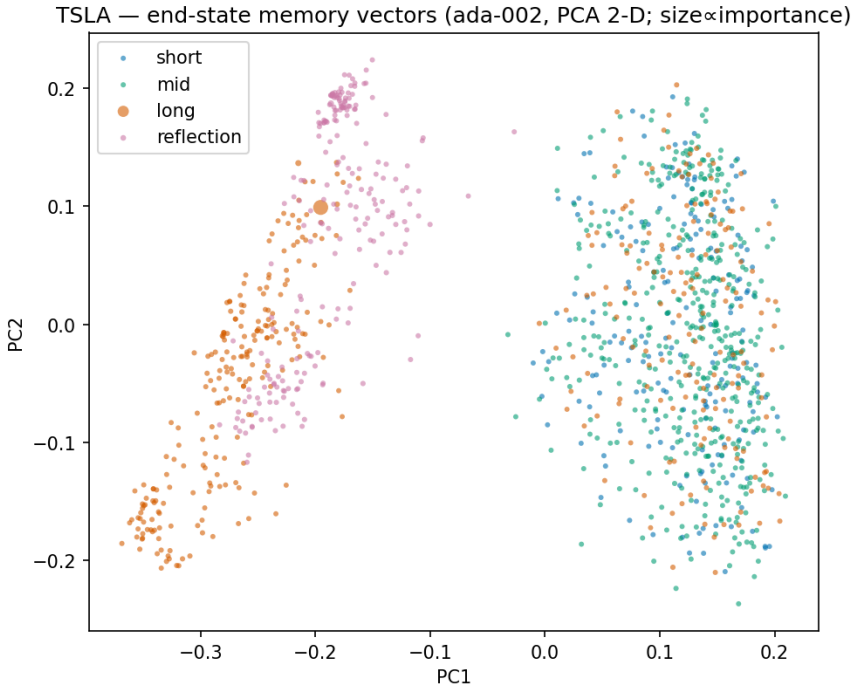

Fig 5. End-state memory vectors (ada-002, PCA), coloured by layer.

8. Key findings

F1 — Pure momentum following. The as-shipped agent agreed with 3-day price momentum on 100% of its trading decisions; our corrected version lowered this to 74%, but performance did not improve.

F2 — The deep memory does not retain. Replaying the agent's own per-day memory logs, every item that entered the deep layer was expelled within three days, and not one SEC filing was ever retained — so the architecture's headline 'long-term memory' claim is not realised by the shipped settings. Our retention change fixes this (the deep layer ends with over a thousand items, and the agent even cites its own past reflections).

F3 — Memory subtracted value. The no-memory ablation beat the full FinMem-Ours on the mean (+1.7% vs –5.3%) and on 3 of 5 tickers.

F4 — A plain long-context model is a stronger baseline. LC-Trader was almost entirely passive (held on roughly 97% of days), tracked the market, and beat FinMem-Ours on the mean (–2.9% vs –5.3%). Its cost, however, grows with the trading horizon as the context lengthens — an overhead the fixed-size memory design avoids.

9. Reproducibility

Every result is regenerable from saved decision logs with no further model spend. The repository contains the data pipeline, the train/test runner, the LC-Trader baseline, the audited metrics module, the figure scripts, and leakage / behaviour test suites. A token-cost meter with hard caps and checkpoint-resume across daily quota resets is built in — the overnight run even survived a mid-run PC crash with no lost work.

Appendix — the original code vs what the paper describes

A by-product of the reproduction: where the published code differs from the paper's text.

Aspect	Paper / README describes	What the shipped code actually does
Risk persona	Self-adaptive (seeking / averse)	Static one-sided line only
Deep memory	Retains information over the long run	Discards within ~3 days; no filings kept
Extended reflection	Daily multi-day self-review	Not implemented
Positions	Long-only narrative	'Sell' opens a costless short
Filings	Inform decisions	Pre-window reports are never loaded